symmetry

MDPI

Article

Real-Time Identification of Strawberry Pests and Diseases Using an Improved YOLOv8 Algorithm

Danyan Xie *, Wenyi Yao, Wenbo Sun and Zhenyu Song *

College of Information Engineering, Taizhou University, Taizhou 225300, China; yaowenyiyi@gmail.com (W.Y.); symb130@gmail.com (W.S.)

* Correspondence: xiedanyan@tzu.edu.cn (D.X.); songzhenyu@tzu.edu.cn (Z.S.)

Abstract: Strawberry crops are susceptible to a wide range of pests and diseases, some of which are insidious and diverse due to the shortness of strawberry plants, and they pose significant challenges to accurate detection. Although deep learning-based techniques to detect crop pests and diseases are effective in addressing these challenges, determining how to find the optimal balance between accuracy, speed, and computation remains a key issue for real-time detection. In this paper, we propose a series of improved algorithms based on the YOLOv8 model for strawberry disease detection. These include improvements to the Convolutional Block Attention Module (CBAM), Super-Lightweight Dynamic Upsampling Operator (DySample), and Omni-Dimensional Dynamic Convolution (ODConv). In experiments, the accuracy of these methods reached 97.519%, 98.028%, and 95.363%, respectively, and the *F*1 evaluation values reached 96.852%, 97.086%, and 95.181%, demonstrating significant improvement compared to the original YOLOv8 model. Among the three improvements, the improved model based on CBAM has the best performance in training stability and convergence, and the change in each index is relatively smooth. The model is accelerated by TensorRT, which achieves fast inference through highly optimized GPU computation, improving the real-time identification of strawberry diseases. The model has been deployed in the cloud, and the developed client can be accessed by calling the API. The feasibility and effectiveness of the system have been verified, providing an important reference for the intelligent research and application of strawberry disease identification.

Keywords: YOLOv8 algorithm; pest and disease recognition; CBAM; DySample; ODConv

check for updates

Citation: Xie, D.; Yao, W.; Sun, W.; Song, Z. Real-Time Identification of Strawberry Pests and Diseases Using an Improved YOLOv8 Algorithm. *Symmetry* **2024**, *16*, 1280. https://doi.org/10.3390/sym16101280

Academic Editors: Zhixun Su and Sergei Odintsov

Received: 3 August 2024
Revised: 4 September 2024
Accepted: 26 September 2024
Published: 29 September 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The scale of strawberry cultivation in China has been expanding year by year, with a strawberry planting area of about 1600 square kilometers as of 2023, and an output value of about 100 billion yuan, ranking first in the world. The growing process sees the occurrence of a variety of pests and diseases, which have a great impact on yield and quality. Strawberry pests and diseases result in poor growth, leaf damage, and hindrance of photosynthesis, which cause wilting, yellowing, and falling of leaves, reducing the plant's nutrient absorption and energy accumulation, which will affect the yield. Strawberry pests and diseases cause problems such as spotting, rotting, and discoloration on the surface of the fruit, which affects its appearance and taste, and cause deformation, breakage, and internal rotting, which affects the taste and quality. To ensure yield and quality, fruit farmers must expend significant energy to monitor the growth of strawberries, and when a large area is planted, there is a need to effectively reduce the cost of labor. The correct identification of strawberry pests and diseases has certain requirements, as the failure to accurately identify relevant pests and diseases in a timely manner can result in delayed treatment or an excess application of pesticides compared with the standard, affecting the environment and increasing pesticide residues. It is therefore imperative that a method be developed to diagnose and classify strawberry pests and diseases with a rapid detection speed and high recognition accuracy.

This study examines the trade-off between accuracy, speed, and computation in the process of identifying pests and diseases affecting strawberries through computer vision. To this end, we propose three improved models based on the YOLOv8 algorithm: the Convolutional Block Attention Module (CBAM) attention mechanism, Lightweight Dynamic Sampling Operator (DySample), and Omni-Dimensional Dynamic Convolution (ODConv). To optimally utilize the data, the training process incorporates K-fold validation. Comparative experiments were conducted to evaluate the effectiveness of the models in terms of precision, recall, *F*1 value, and *MAP*. The loss is analyzed to select the model with the best stability and convergence. In our experiments, the CBAM attention mechanism and DySample showed slight improvements. Among these, LOSS, which incorporates the CBAM attention mechanism, demonstrated superior stability and convergence. Tensor Routing and Transposing (TensorRT) acceleration is employed to achieve rapid inference through optimized GPU computation. The experimental results demonstrate that the accelerated inference time achieved by TensorRT is reduced to approximately one-third of the original. The model developed in this paper has been deployed in a cloud server, and a mobile phone client that is straightforward for farmers to utilize has been created. The strawberry pest and disease recognition model was successfully deployed in a real-world production setting, demonstrating both operational feasibility and practical utility. The main contributions of this work are as follows:

- The CBAM attention mechanism is incorporated into the YOLOv8 network. This involves the Channel Attention Module compressing the spatial information of each channel into a single value, which is achieved by performing maximum and average pooling on the input feature maps. This allows for the compression and extraction of global spatial information. The spatial attention module identifies the relative importance of input data in different spatial locations. By learning the weights associated with these, the model can allocate more attention to those deemed important, thereby enhancing overall performance and generalization.
- DySample is incorporated into the YOLOv8 network. This Super-Lightweight Dynamic Upsampling Operator can achieve efficient upsampling with low computational resources, reducing computation and storage through differential sampling.
- The multidimensional convolutional kernel, ODConv, adopts a multidimensional attention mechanism and parallelization to facilitate the learning of complementary attention in all four dimensions of the convolutional kernel space. A comprehensive multidimensional attention mechanism enhances the dynamic convolutional approach, improving performance and efficiency across a range of convolutional neural network (CNN) architectures.
- TensorRT acceleration facilitates inference through optimized GPU computation, markedly reducing the overhead associated with memory transfer and storage. The inference process is enhanced through mixed-precision inference while maintaining the requisite degree of accuracy. Low-precision computations, including FP16 and INT8, are utilized to accelerate the inference process while maintaining the accuracy of the model and ensuring real-time performance.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 introduces the materials, including the YOLOv8 model and LOSS. Section 4 describes methods, focusing on enhancements such as the CBAM attention mechanism, DySample, and ODConv. Section 5 describes our experiments, including the datasets and parameter settings, and discusses the results. Section 6 presents our conclusions and suggests future work.

2. Related Work

Recent years have seen work on the use of machine learning techniques to detect pests and diseases in crops [1]. Nevertheless, conventional machine vision techniques encounter certain limitations in the identification of pests and diseases in diverse crops, particularly in the presence of complex and disrupted image scenes, which cannot satisfy the

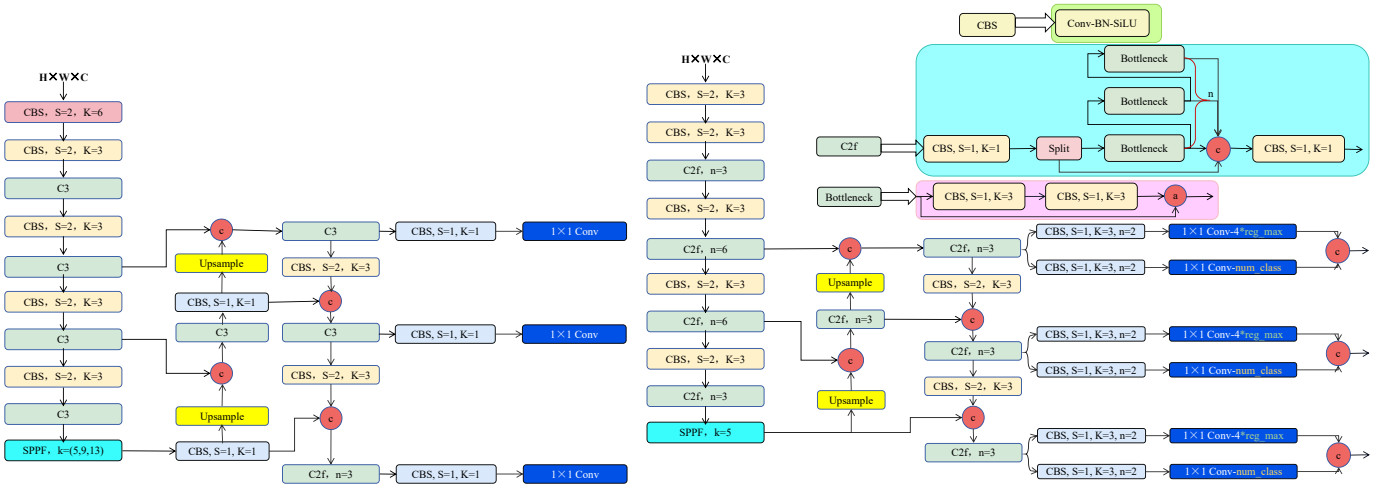
requirements of real-time object detection. With the advent of deep learning, in particular, the successful application of CNNs, deep learning models have made significant progress in areas such as computer vision [2] and natural language processing [3]. In the field of plant disease detection, researchers have used a variety of deep learning architectures, such as AlexNet [4], GoogleNet [5], the Visual Geometry Group Network (VGGNet) [6], the Residual Network (ResNet) [7], and Vision Transformer [8], which take advantage of the CNN's deep feature extraction and integrated models to accurately localize and classify plant pests and diseases. Ashwini et al. used hybrid 3D-CNN and LSTM models for maize leaf disease recognition [9]. Zhang et al. proposed a multi-scale feature fusion instance detection method that improves the detection of maize leaf blight in complex backgrounds [10]. While these methods are useful, they cannot meet the needs of real-time object detection. The YOLO algorithm has also been widely used for pest detection [11], and researchers have optimized and improved it for use in complex environments. Roy et al. improved Dense-YOLOv4, a real-time object recognition system, by integrating a Dense Convolutional Network (DenseNet) into the backbone network to optimize feature transfer and reuse [12]. Appe et al. used improved tomato detection and classification based on YOLOv5 (using a combined attention mechanism) [13]. Tian et al. proposed a deep learning-based method for apple anthracnose detection, using DenseNet to optimize the low-resolution feature layer of the YOLOv3 model, which improved the utilization of neural network features and detection results [14]. In conclusion, the YOLO algorithm addresses the end-to-end real-time detection problem by transforming the object detection task into a regression task, and directly classifying it using global or local features. The rapid detection capability of the YOLO algorithm makes it a promising candidate for a wide range of applications in the field of real-time identification of plant pests and diseases.

In addressing the problems of small strawberry plants, and the hidden nature and variety of pests and diseases, it is difficult to balance between accuracy, speed, and computation using the above-mentioned YOLO series. However, effective work has been carried out on pest and disease recognition models for strawberries. Wani et al. [15], Singh et al. [16], Griffel et al. [17], and Kartikeyan et al. [18] used traditional machine learning techniques to identify crop diseases. The advent of deep learning has prompted an increasing number of researchers to incorporate inference engines into crop disease recognition algorithms, thereby enhancing the speed of recognition. Yu et al. [19] proposed an adaptive algorithm based on deep residual neural networks, with better results. Abbas et al. [20] used CNNs to classify and identify strawberry pests and diseases, using migration learning to accelerate model training and improve recognition accuracy. Pérez-Borrero et al. [21] proposed a segmentation method for strawberry instances based on deep learning techniques, using a new dataset to train the neural network, with better results. Guo et al. [22] combined self-supervised multi-network fusion classification models in a lossless and convenient method, which improved the efficiency of strawberry disease identification. In this paper, we propose an improved YOLOv8 detection algorithm suitable for strawberry pest recognition, combining the advantages of the YOLO algorithm and the characteristics of strawberry pest images. YOLOv8 [23] integrates a large number of optimization strategies, making extracted features prone to the influence of noise, insufficient feature information, and other problems, which can result in misdetection and wrong detection. Our new model introduces some of the effective improvements of the YOLO family of crop disease detection into YOLOv8-based strawberry disease detection [24–26]. The YOLOv8 model has been enhanced by incorporating the CBAM attention mechanism, DySample, and ODConv. The appropriate balance of accuracy, speed, and computation in real-time strawberry pest and disease detection is addressed through multiple model comparisons and evaluations, along with TensorRT acceleration.

3. Materials

3.1. Principles of the YOLOv5 and YOLOv8 Algorithms

The YOLOv5 and YOLOv8 algorithms represent two classic members of the YOLO family of algorithms, whose network architecture is illustrated in Figure 1. The YOLOv5 algorithm is characterized by high processing speed and a compact model. The YOLOv8 algorithm incorporates a number of enhancements over the YOLOv5 algorithm, with the objective of improving accuracy. YOLOv8 represents the latest iteration of the YOLO family of object detection algorithms. The algorithm is employed in a variety of applications, including image classification, object detection, and instance segmentation. It incorporates novel features that enhance the performance and flexibility of the model. It is regarded as one of the most effective options for tasks such as object detection, image segmentation, and pose estimation. YOLOv8 offers new state-of-the-art (SOTA) models, including the P5 640 and P6 1280 resolution networks for object detection, as well as a YOLACT-based instance segmentation model. Furthermore, YOLOv8 offers models based on diverse scaling factors, including YOLOv8s, YOLOv8n, YOLOv8l, and YOLOv8x, thereby catering to a variety of requirements.



(**a**) YOLOv5 network architecture. (**b**) YOLOv8 network architecture.

Figure 1. Principles of the YOLOv5 and YOLOv8 algorithms.

3.1.1. Differences between YOLOv8 and YOLOv5 Network Structures

The YOLOv8 model has undergone a significant number of optimizations and improvements from the YOLOv5 network structure. Backbone: YOLOv8 replaces the C3 module in YOLOv5 with the C2f module for further lightweighting. Figure 2 illustrates the difference between C3 and C2f modules.The SPPF module in YOLOv5 is followed, and the model is carefully fine-tuned for different scales, resulting in a substantial improvement in model performance, as shown in Figure 2. The kernel size of the initial convolutional layer is modified from 6×6 to 3×3. All C3 modules have been replaced with C2f modules, which feature additional layer-hopping connections and split operations. The number of blocks has been modified from 3-6-9-3 in C3 modules to 3-6-6-3 in C2f modules. The number of channels in the input tensor of each bottleneck in C2f is halved from the previous level, resulting in notably less computation, and an increase in gradient flow can enhance the convergence speed and effect. The C2f module initially processes the input tensor (n, c, h, w) through the Conv1 layer for splitting, dividing it into two parts, $(n, 0.5c, h, w)$. One part directly passes through the n bottleneck, while the other undergoes a series of operations, ultimately outputting through the Conv2 layer convolution. The input tensor, with subscripts (c, h, w), is split into two parts, with one passing directly through the n bottleneck, and the other undergoing a shortcut operation with a size of $(n, 0.5c, h, w)$. This process continues through each operation layer, ultimately resulting in output through the

Conv2 layer. This is equivalent to an $n + 2$ shortcut (the branch tensor of the initial Conv1 layer and the tensor resulting from splitting are both $n + 2$ bottlenecks).

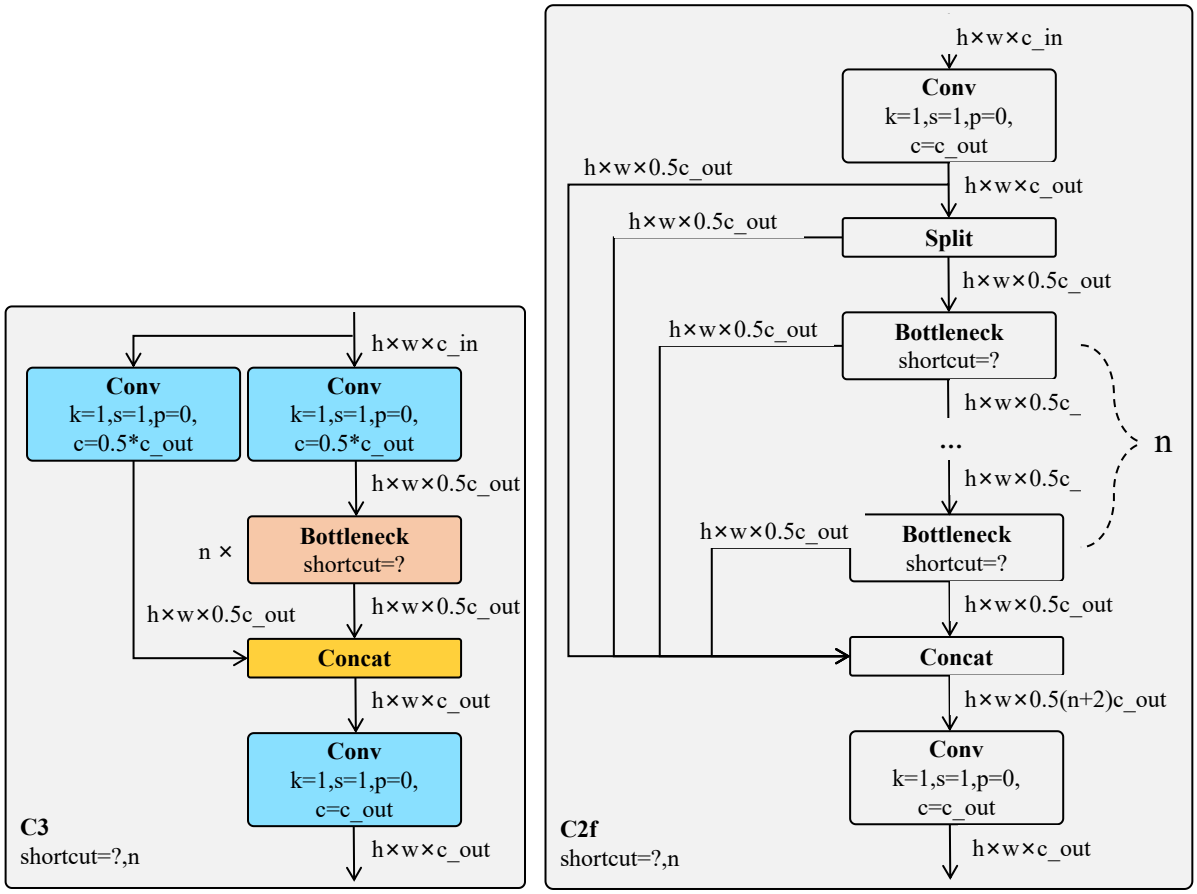


Figure 2. Difference between C3 and C2f modules.

Neck: The concept of PAN is utilized, the structure of which is based on downsampling from the bottom–up. YOLOv8 eliminates the $1 * 1$ downsampling layer, as illustrated in Figure 3. Subsequently, the width and height are downsampled 32 times following the final SPPF module of the backbone (Layer 9). This is equivalent to layers 4 and 6 being downsampled 8 and 16 times, respectively. The input image resolution is 640 by 640 pixels, and the resolutions of layers 4, 6, and 9 are 80 by 80 pixels, 40 by 40 pixels, and 20 by 20 pixels, respectively. These layers are used as inputs to the PANet structure, which is upsampled, and channel-fused, and the three output branches of PANet are fed into the Detect head for loss calculation or result-solving. In contrast to FPN (unidirectional, top–down), PANet is a bidirectional pathway network that incorporates bottom–up paths, facilitating the transfer of information from the bottom layer to the top layer.

Head: The head component has undergone significant modifications from YOLOv5. YOLOv8 adopts the current mainstream Decoupled-Head (DH) structure, which separates the regression branch from the prediction branch. It utilizes integral form representation, as proposed in the Distribution Focal Loss strategy for the regression branch. While previous target detection networks predicted the regression coordinate as a deterministic single value, DFL transforms the coordinate into a distribution. The difference between YOLOv5 and YOLOv8 in the Head section is illustrated in Figure 4.

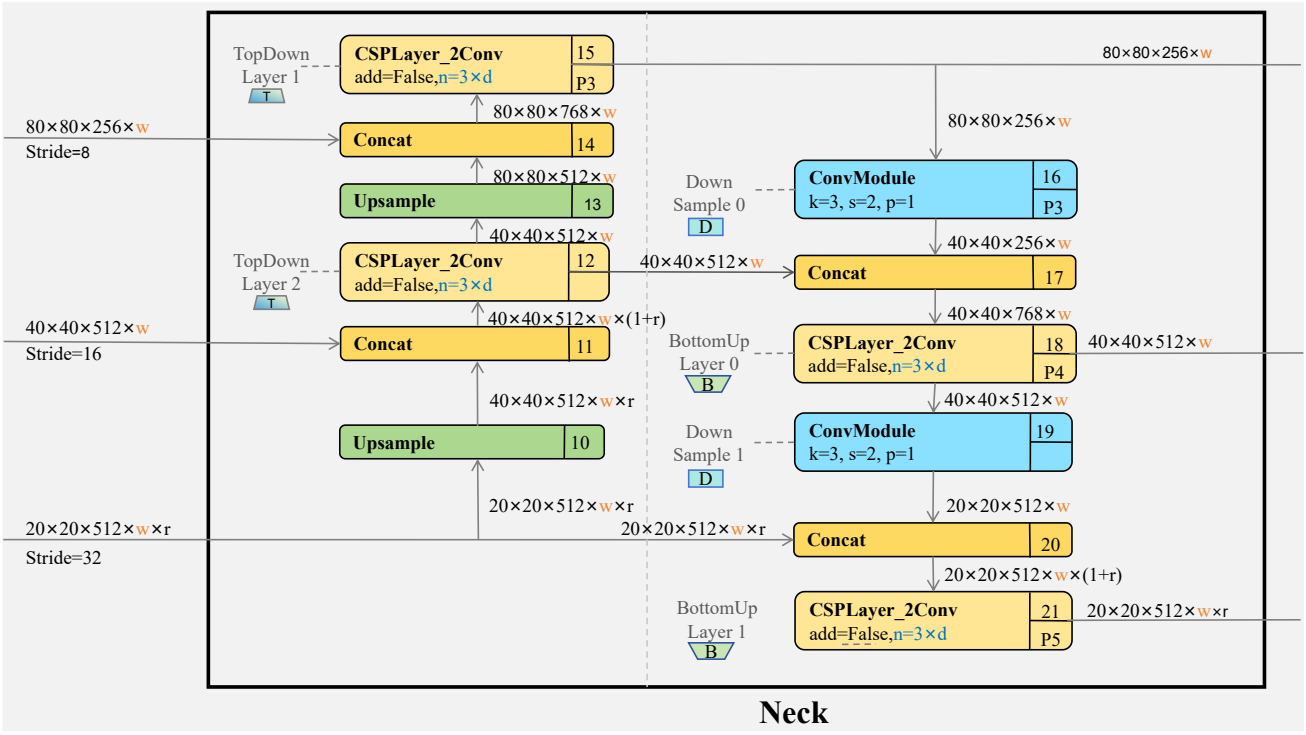


Figure 3. Neck structure.

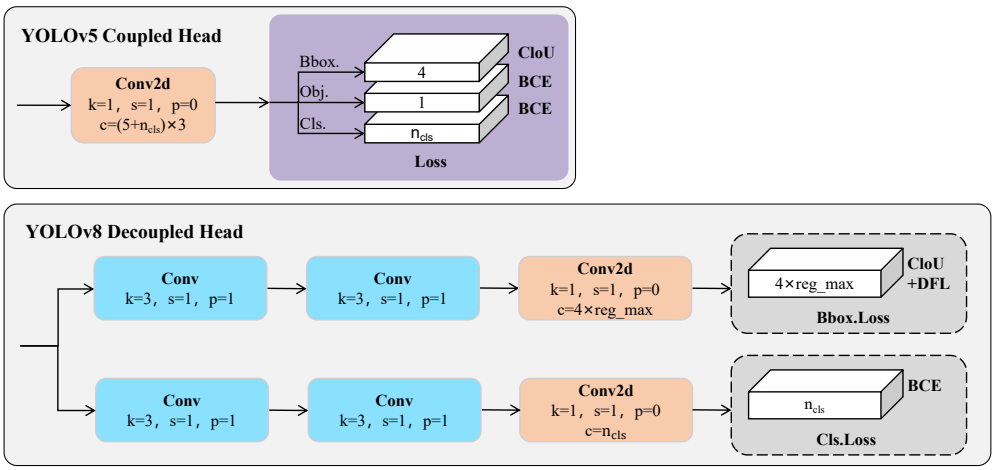


Figure 4. Difference between YOLOv5 and YOLOv8 in Head section.

3.1.2. Loss Calculation of YOLOv8

The loss calculation is divided into two distinct phases: the initial allocation of positive and negative samples and the subsequent loss calculation. The most prevalent approaches to positive and negative sample allocation are the dynamic and static allocation strategies. A static allocation strategy entails the determination of a set of predefined weights prior to the training phase, which remain constant throughout. This approach is typically derived through empirical means, and it can be adapted to align with the specific characteristics of the dataset. This approach is not sufficiently flexible and may not fully utilize the sample information, which can result in suboptimal training outcomes. In contrast to the static allocation strategy, the dynamic allocation strategy allows the weights to be adjusted according to the training process and sample characteristics. In the initial phase of the training process, the model may experience challenges in accurately distinguishing between positive and negative samples. It is therefore crucial to prioritize samples that are susceptible to misclassification. As the training process continues, the model's capacity

to differentiate between samples improves. It is therefore recommended that the weights assigned to the most challenging samples be reduced, while those assigned to samples that are less difficult to classify be increased. The dynamic allocation strategy may be modified based on training losses or other metrics, thus facilitating adaptation to different datasets and models.

Notable examples of dynamic allocation strategies include simOTA in YOLOX, Task-Aligned Assigner in TOOD, and DynamicSoftLabelAssigner in RTMDet. YOLOv5 continues to utilize a static allocation strategy. In light of the superior performance demonstrated by dynamic allocation strategies, the YOLOv8 algorithm directly references the Task-Aligned Assigner positive and negative sample allocation strategy developed by TOOD. The ratio of positive to negative samples is adjusted dynamically throughout the training process. The positive samples are selected on the basis of weighted classification and regression scores:

$$t = s^{\alpha} + u^{\beta}, \tag{1}$$

where s is the prediction score corresponding to the labeled category, and u is the IoU of the Prediction Box and GT Box, which are multiplied to measure the degree of alignment (Task-Alignment). α and β are weight hyperparameters. t can simultaneously control the optimization of categorical scores and IoU to achieve Task-Alignment, thus guiding the network to dynamically focus on high-quality anchors. As the category scores and IoU are higher, the value of t will be closer to 1. Loss computation consists of categorization and regression branches [27,28]. Categorical loss uses a sigmoid function to compute the probability of each category, and VFL or BCE Loss to compute the global categorical loss.

- VFL Loss Focal Loss is designed to solve the problem of extreme imbalance between foreground and background classes in the training of dense target detectors:

$$FL(p, y) = \begin{cases} -\alpha(1-p)^{\gamma}\log & if \quad y = 1 \\ -(1-\alpha)p^{\gamma}\log(1-p) & otherwise \end{cases}, \tag{2}$$

  where α is the weighting factor, γ is the moderator, p is the probability of the true category, and 1 is the ground-truth class and represents the predicted probability of the foreground class. The modulation factor serves to diminish the impact of loss contributions associated with simple samples while concomitantly elevating the significance of misclassified samples. In contrast to Focal Loss, which treats positive and negative samples in a symmetrical manner, VFL Loss introduces an asymmetric weighting operation defined as follows:

$$VFL(p, q) = \begin{cases} -q(q\log(p) + (1-q)\log(1-q)) & \text{when} \quad q > 0 \\ -\alpha p^{\gamma}\log(1-p) & \text{when} \quad q = 0 \end{cases}, \tag{3}$$

  where the predicted value is the target score. For foreground points, we set their ground-truth class score to the IoU between the generated bounding box and its $groundtruth(gt_IoU)$, or 0 otherwise; and for background points, the scores for all classes are 0. The VFL Loss utilizes the factor scaling loss of γ, which serves to reduce the loss contribution of only the negative examples $(q = 0)$ without simultaneously reducing the weight of the positive examples $(q > 0)$.
- BCE Loss YOLOv8 uses Binary Cross-entropy (BCE) Loss, which is the loss function used for binary classification:

$$L(pt, target) = -w * (target * ln(pt) + (1 - target) * ln(1 - pt)), \tag{4}$$

  where pt is the model prediction value, $target$ is the label value, and w is the weight, which is usually 1. This is for a single sample. When there are N samples in a batch,

$$\text{loss} = \frac{1}{N}\sum L. \tag{5}$$

4. Methods

To find a suitable model for strawberry pest and disease detection, we propose improved models based on the YOLOv8 algorithm with CBAM, DySample, and ODConv. The method flow is shown in Figure 5.

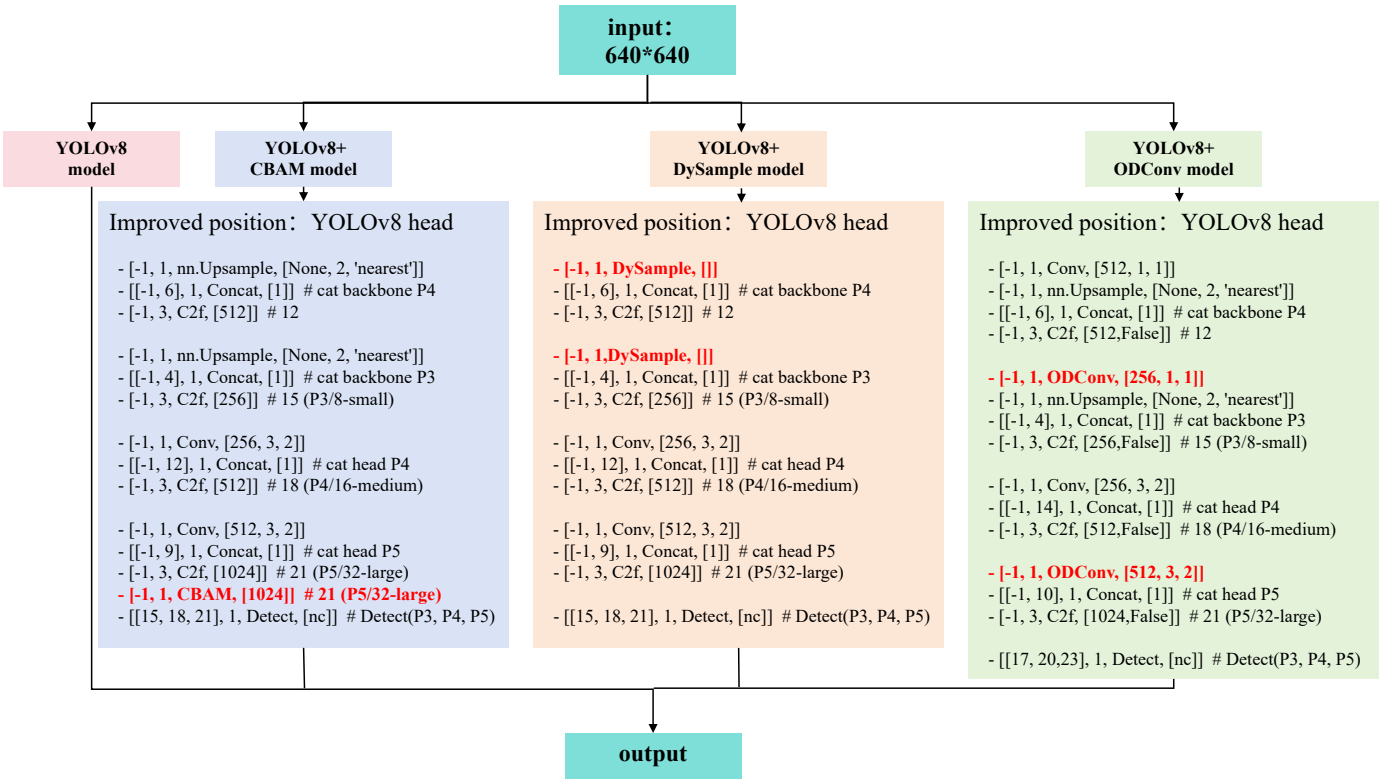


Figure 5. Different improvements to the YOLOv8 algorithm.

4.1. Improvement of YOLOv8 Algorithm Based on CBAM

An attention mechanism is a computational technique that prioritizes localized information, thereby directing the system's attention toward the focused information. This mechanism plays a pivotal role in tasks such as image, object, and face recognition. The implementation of the attention mechanism may be achieved through a number of approaches, including the use of a spatial attention model, channel attention model, or hybrid spatial and channel attention model. These models are capable of extracting the key information from an image, thereby enhancing overall model performance by suppressing superfluous information. The introduction of an attention mechanism enables a computer vision system to more efficiently process image data, thereby reducing computation, while enhancing performance and accuracy.

The Convolutional Block Attention Module (CBAM) is an attention mechanism that combines channel and spatial attention, thereby enhancing the performance of convolutional neural networks [29]. The Channel Attention Module distinguishes between the features present in different channels by calculating the relative importance of each channel. The spatial attention module determines the spatial importance of each pixel, facilitating the capture of the spatial structure of the image. CBAM comprises a Channel Attention Module (CAM) and Spatial Attention Module (SAM), which, respectively, perform channel and spatial attention. This reduces the number of required parameters and computational resources and allows for seamless integration into existing network architectures as a plug-and-play module, as illustrated in Figure 6.

As illustrated in Figure 7, the Channel Attention Module outputs a channel attention weight vector through the application of maximum and average pooling techniques on the

input feature map in the channel dimension. These two pooling results are fed into a fully connected layer. The weight vector can be employed to assign weights to each channel of the input feature map, thereby enhancing salient channel features and suppressing those that are unimportant.

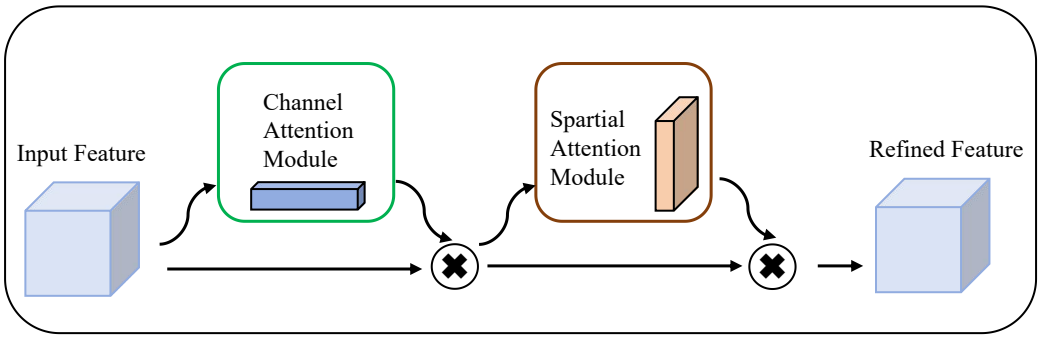


Figure 6. Convolutional Block Attention module.

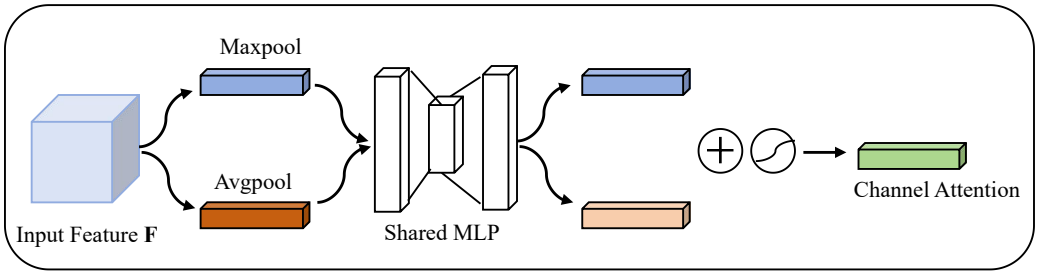


Figure 7. Channel Attention mechanism.

The fundamental objective of the CAM is to highlight the most pertinent feature channels in relation to the task at hand. The aforementioned elements are as follows:

- The input feature map X, of shape (B, C, H, W), is subjected to feature compression through maximum and average pooling operations, both performed on the spatial dimension, resulting in two feature maps of shape $(B, C, 1, 1)$, which entails the compression of each channel into a single value representing the maximum and average, respectively.
- Dimensionality transformation is achieved through the use of a small neural network, typically a two-layer MLP, which downscales the number of channels to reduce the number of parameters and then upscales them back to the original number of channels. The network comprises two fully connected layers and a ReLU activation function.
- The two feature maps, obtained from maximum and average pooling, are processed through a shared multi-layer perceptron (MLP). The results are summed and passed through a sigmoid function, which yields the weight coefficients for each channel.

The spatial attention module, illustrated in Figure 8, is analogous to the CAM in that both modules compute the importance of each pixel by manipulating the input feature map. Typically, the module employs global average pooling to derive a feature vector for each pixel, subsequently outputting their weights through a fully connected layer. These weights can be applied to each pixel of the input feature map, thereby emphasizing important regions in the image and suppressing unimportant regions.

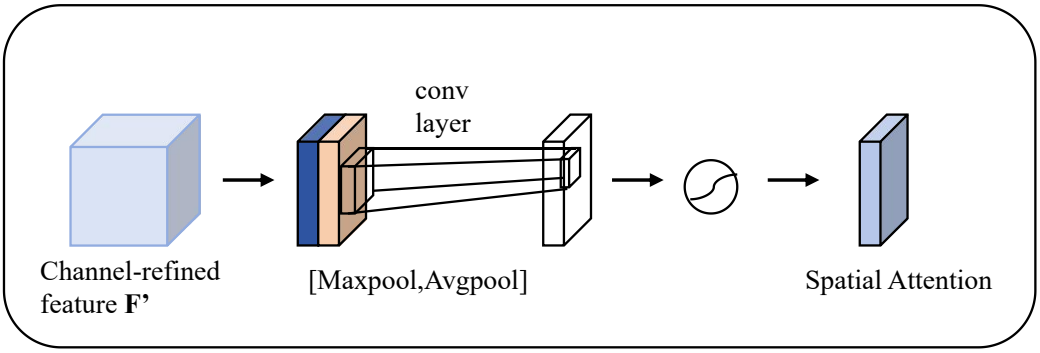


Figure 8. Spatial Attention mechanism.

The purpose of SAM is to spatially emphasize more critical areas, as follows:

- In the context of channel compression, the processed feature map X is subjected to maximum and average pooling operations, this time along the channel axis C. This results in the compression of all channel information into a single-channel image. The result of the operation is two feature maps of shape $(B, 1, H, W)$.
- The two feature maps are spliced in the channel dimension, after which the number of channels is modified to one through the application of a convolutional layer. Ultimately, the weight coefficients associated with each spatial location are obtained through the utilization of a sigmoid function.

The Channel Attention and Spatial Attention Modules form a complete CBAM module, which can be integrated into a Convolutional Neural Network. The significance of each channel is recalibrated by multiplying the channel weights derived from the Channel Attention Module by the original feature map X. The adjusted feature map is fed into the Spatial Attention Module, where it undergoes further adjustments to the importance of each position. The application of channel attention followed by spatial attention is typically more effective because, once the most important feature channels have been identified, adjusting the importance of each position in these channels allows for a more accurate reinforcement of useful information and suppression of unnecessary information. The application of the CBAM can notably enhance the performance of computer vision tasks, including target detection, image classification, and semantic segmentation.

4.2. Improvement of YOLOv8 Algorithm Based on DySample

DySample [30] is an efficient dynamic upsampling operator that achieves excellent results with limited computational resources. The operator adopts Differential Sampling (DS), which markedly reduces the computational and storage requirements by selecting the portions of the data distribution exhibiting significant disparities for sampling. In particular, the DySample operator determines the discrepancy between the current pixel and its neighboring pixels in each sampling phase, selecting only those with a greater difference for sampling, reducing the amount of data and the computational complexity. We propose the replacement of the UpSample operator with DySample to improve the YOLOv8 algorithm, as illustrated in Figure 9.

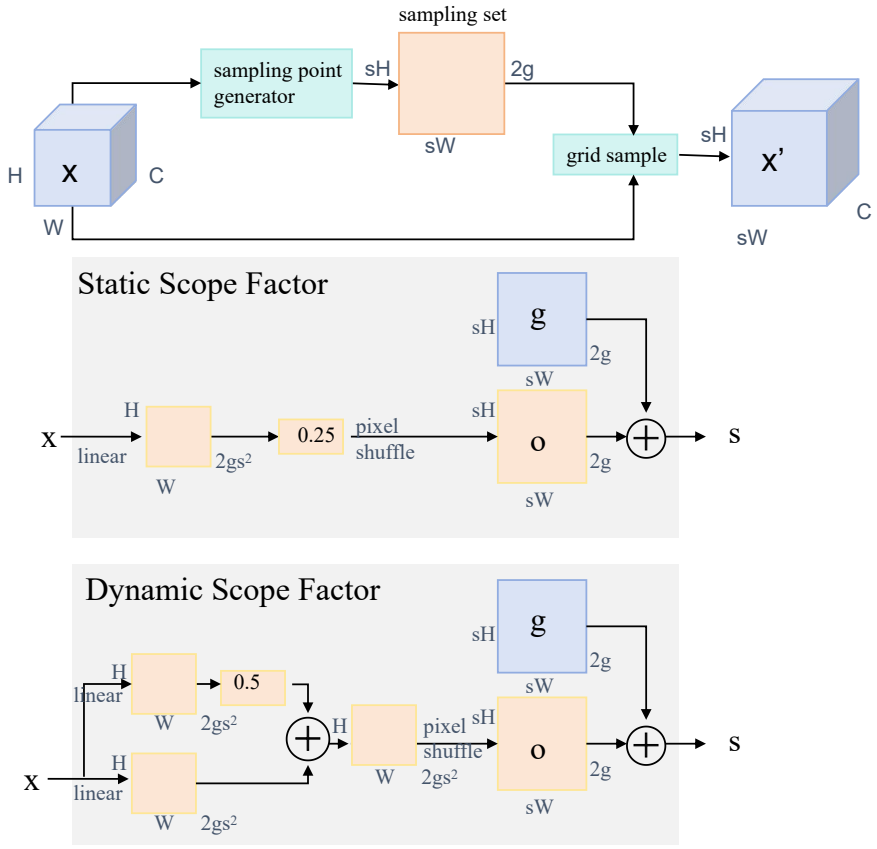


Figure 9. Improvement of YOLOv8 algorithm based on DySample.

4.3. Improvement of YOLOv8 Algorithm Based on ODConv

ODConv [31] is a full-dimensional dynamic convolutional design that uses a multi-dimensional attention mechanism and a parallel strategy. It is capable of learning complementary attention in all four dimensions of the space of convolutional kernels of a convolutional layer, i.e., the spatial size of each convolutional kernel, and the number of input channels, output channels, and convolutional kernels. The objective is to learn complementary attention. In the YOLOv8 model, ODConv is employed in the C2f and bottleneck modules in lieu of standard convolution, which facilitates greater accuracy and efficiency in image recognition, as illustrated in Figure 10.

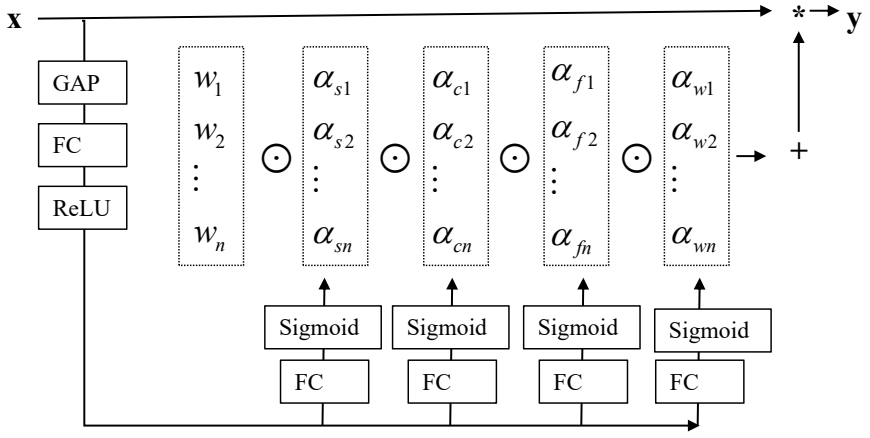


Figure 10. Improvement of YOLOv8 algorithm based on ODConv.

5. Experimental Results and Discussion

5.1. Experimental Setup

The experimental environment selected for this study was the Ubuntu 22.04 operating system, an Intel Xeon Platinum 8474C central processing unit (CPU), and an NVIDIA RTX4090 graphics processing unit (GPU), with Python version 3.8.10, PyTorch version 2.0.0, and CUDA version 11.8. The experimental procedure had the following steps:

- Data collection and preprocessing: It is necessary to assemble image datasets comprising disparate strawberry pest and disease samples, which are divided into training and testing sets. It is recommended that the pest and disease datasets be maintained in its original form, minimizing preprocessing, to more accurately reflect the circumstances in the field, with some types of pests and diseases occurring more or less frequently.
- Model training: Training used four distinct network models—the original YOLOv8 and the integration of the CBAM attention mechanisms, DySample, multidimensional convolutional kernels, and ODConv—into the YOLOv8 network. The appropriate parameters were set, such as the learning rate and batch size. The model was optimized using suitable optimization algorithms to monitor the training process and record metrics such as training loss and accuracy. We sought to identify an appropriate and efficient model through comparative experimentation.
- Model evaluation: The recognition accuracy of the model was determined through the accuracy, recall, and $F1$ score, and real-time performance was determined by calculating its inference speed. A comparative analysis was conducted between the enhanced and conventional YOLOv8 algorithms.

5.2. Evaluation Metrics

In this experiment, the performance of the detection algorithm was evaluated by the precision, recall, $F1$ score, mAP@0.5, and mAP@0.5:0.95. The $F1$ score is a composite metric that considers both precision and recall to evaluate the performance of the algorithm on positive and negative samples. The mean average precision (mAP) is used to assess the accuracy of multi-category target detection. It is the average of all category-specific average precision (AP) values, providing a measure of the model's overall performance. A confusion matrix can assess the efficacy of a classification model and is presented in a tabular format, as illustrated in Table 1, where True Positive (TP) indicates a correctly predicted positive case, False Negative (FN) indicates a positive case predicted as negative, False Positive (FP) indicates a negative predicted as positive, and True Negative (TN) indicates a correctly predicted negative case.

Table 1. Representation of the confusion matrix.

	Positive	**Negative**
Positive	True Positive	False Negative
Negative	False Positive	True Negative

The precision, or accuracy rate, indicates the percentage of samples correctly identified as positive by the trained model. In general, a higher accuracy rate indicates a better model:

$$Precision = \frac{TP}{TP + FP} \times 100\%. \tag{6}$$

Recall, or the check all rate, indicates how many actual positive samples are predicted by the classifier. A higher value indicates a better model:

$$Recall = \frac{TP}{TP + FN} \times 100\%. \tag{7}$$

The $F1$ score combines precision and recall and can be viewed as their reconciled average. Its maximum value is 1, and the minimum is 0. Higher values imply better model

performance. When dealing with unbalanced datasets or with different tolerances for false positives and negatives, the $F1$ score can provide a comprehensive assessment, as it considers both precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \times 100\%. \quad (8)$$

Average precision (AP) is the area below the accuracy and recall curves, while mean average precision (mAP) is the average AP across all categories:

$$mAP = \frac{\sum_{i=1}^{c} AP_i}{C}, \quad (9)$$

mAP@0.5 is the mean average accuracy across all categories when the IoU threshold is 0.5, and mAP@0.5:0.95 is the mean average accuracy when the IoU threshold ranges from 0.5 to 0.95 in steps of 0.05. mAP@0.5:0.95 considers the performance of the model under several IoU thresholds for a more comprehensive evaluation, focusing on whether the model can roughly locate the target and also whether it can precisely locate it.

5.3. K-Fold Validation

K-fold cross-validation (K-fold) [32] is frequently used to assess machine learning models on training data. The original dataset is divided into K subsets, which are referred to as "folds". The model is trained and validated on K occasions, each time utilizing a distinct training set. In each iteration, one fold is designated as the validation set, and the remaining K-1 folds constitute the training set. This yields K models and their corresponding validation scores. The methodology of K-fold cross-validation is illustrated in Figure 11.

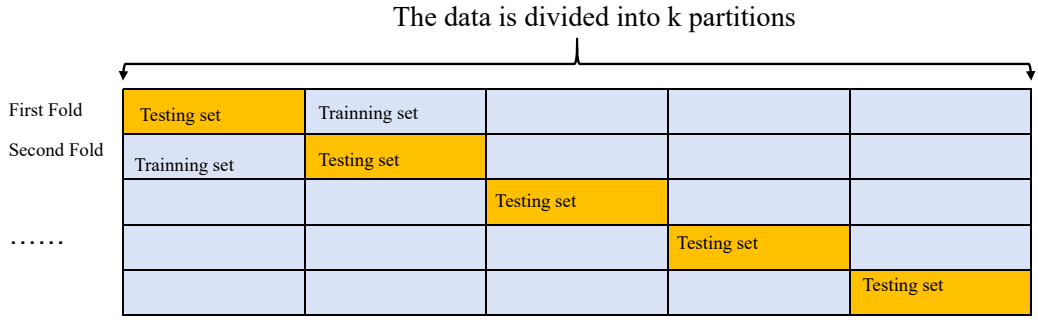


Figure 11. K-fold cross-validation.

The model is then trained and evaluated on the validation set, and the resulting metrics are recorded. These steps are repeated until each subset has acted as a validation set once, resulting in K evaluation metrics, whose mean is taken as the final value. K-fold cross-validation is an effective method, which can reduce overfitting and provide a more reliable assessment of a model's ability to generalize. However, the method has certain disadvantages, including a higher computational cost. As the training and validation sets are divided differently each time, the results of model evaluation may be subject to some volatility. K-fold cross-validation facilitates the selection of the most appropriate model for a particular task by enabling the comparison of the performance of different models on the same dataset. By plotting receiver operating characteristic curves and calculating area under the curve (AUC) values, as well as precision, recall, and other metrics, it is possible to comprehensively evaluate the relative strengths and weaknesses of different models.

5.4. Dataset Description

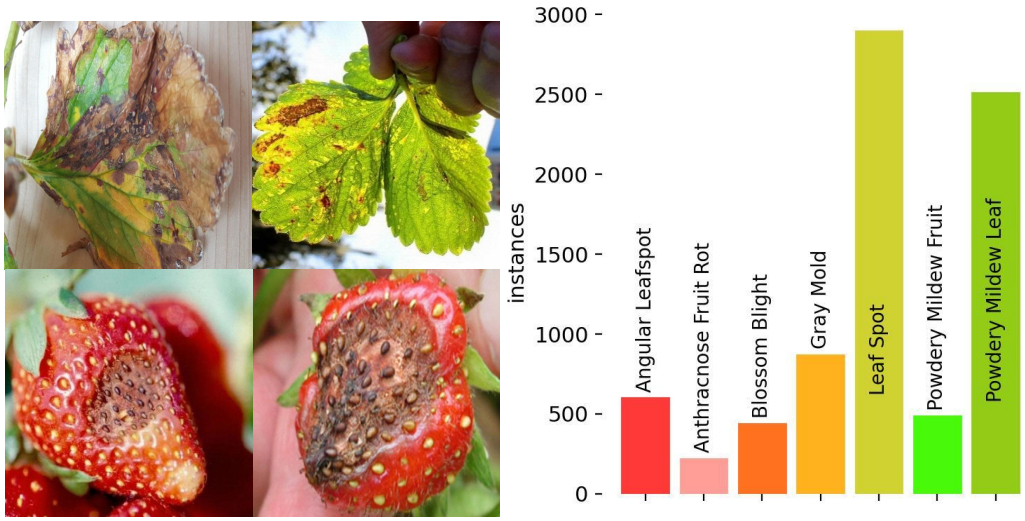
In the absence of a dedicated strawberry pest and disease dataset in the public domain, we undertook a photographic survey of 7980 images of various strawberry pests and diseases. The images were grouped into seven categories of strawberry pests and diseases,

and their labels were confirmed with strawberry growers to ensure the accuracy of the data. The distribution of the sample is illustrated in Table 2.

Table 2. Sample species and numbers.

Angular Leafspot	Anthracnose Fruit Rot	Blossom Blight	Gray Mold	Leaf Spot	Powdery Mildew Fruit	Powdery Mildew Leaf
620	230	450	880	2800	500	2500

The dataset pertaining to strawberry pests and diseases is illustrated in Figure 12. Figure 12a is a picture of strawberry pests and diseases, while Figure 12b shows a histogram of the sample distribution, including a total of seven identified pest and disease types.



(**a**) The strawberry pest and disease picture. (**b**) The histogram of the sample distribution.

Figure 12. The dataset pertaining to strawberry pests and diseases.

Figure 13 illustrates the manner in which the target detection algorithm represents the correlation between labels during the training phase. The label used to train the model is represented by each matrix cell, whose color reflects the strength of the correlation between the corresponding labels. The presence of dark cells indicates that the model has learned the correlation between the two labels with greater strength. Lighter-colored cells indicate comparatively weaker correlation. The coloration displayed on the diagonal represents the correlation of each label with itself. This is typically the darkest shade, as the model is more likely to learn the relationship between labels and themselves. The identification of strong correlations between labels is of paramount importance for the optimization of training and prediction results. If the correlation between certain labels is deemed to be excessive, it may be advisable to consolidate them to streamline the model and enhance its efficacy [33].

Prior to model training, the dataset was divided into training and validation sets at a ratio of approximately 8:2. Five-fold cross-validation was performed, whereby the entire dataset was divided into five equal parts, each representing approximately 20% of the dataset. The training phase was conducted on four of the five parts, with the remaining part serving as the validation set. This process was repeated five times, and the resulting mean was taken as the final result.

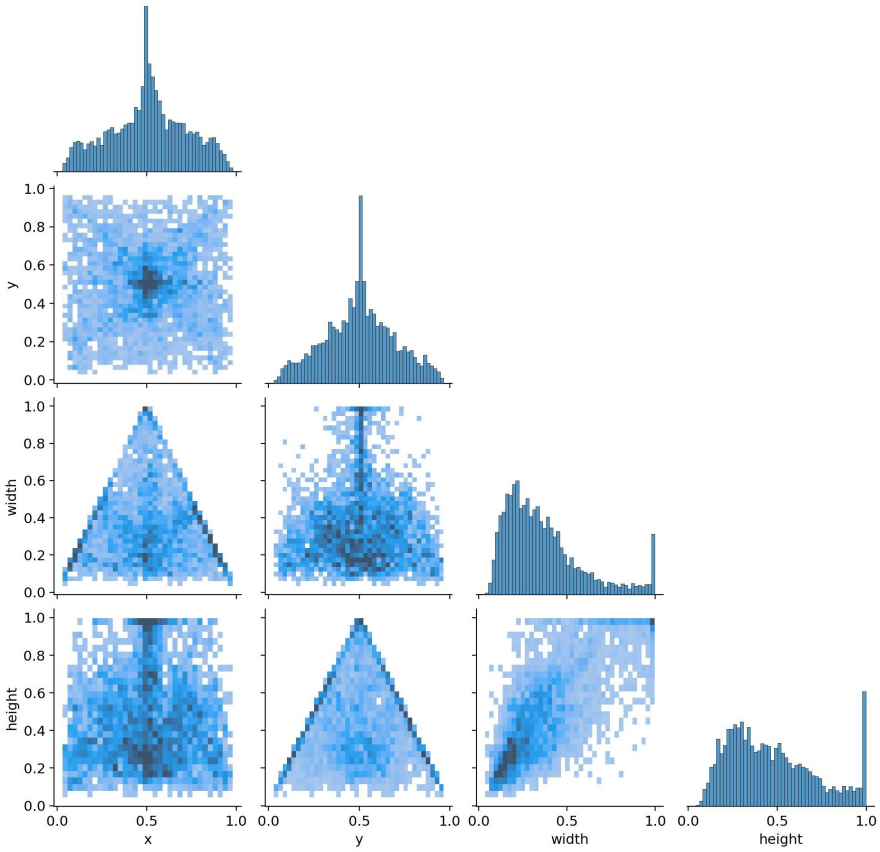


Figure 13. Feature correlation graph.

5.5. Preprocessing

The strawberry pest and disease training set has an image data structure and an annotation data structure. Image preprocessing involves adjusting the image to the requisite size for training the model. The image size was selected as 640 × 640. No data enhancement was performed on the original image, with the objective of more accurately simulating actual farmland pest and disease scenarios. The dataset was labeled using the LabelImg tool.

5.6. Parameter Settings

The YOLOv8 parameters encompass a range of variables, including the learning rate, batch size, input image size, anchor boxes, and regularization. The learning rate controls the speed at which the weights are updated during model training. It is common practice to use a higher initial learning rate and gradually reduce it as training progresses. The batch size defines the number of training samples used to update the weights in each iteration. It is necessary to consider memory constraints and computational resources when choosing a batch size. The input image size defines the size of the image accepted by the model. It is necessary to trade-off between accuracy and speed when choosing an appropriate size. Anchor boxes are used to define targets of varying scales. YOLOv8 utilizes *K*-means clustering to automatically generate anchor boxes based on the training data. The selection of an appropriate number and scale of anchor boxes can enhance the accuracy of detection. Regularization is a technique to prevent overfitting by controlling the complexity of the model. YOLOv8 employs L1 and L2 regularization to limit the size of the model weight. The appropriate regularization can be used to limit the size of the model weight, thereby improving the generalization ability of the model and preventing overfitting. Table 3 illustrates the parameter settings.

Table 3. Parameter selection.

Name	Parameters	Description
Input Image Size	640 × 640	The size of input images
epochs	500	The number of epochs to train for
patience	50	Early parked rounds
batch_size	32	The number of images per batch
iou	0.7	*IoU* thresholds for NMS
max_det	300	Maximum number of detections per image
workspace	4	TensorRT workspace size (GB)
lr0	0.01	Initial learning rate
hsv_h	0.015	Image HSV color tone enhancement
hsv_s	0.7	Image HSV saturation enhancement
hsv_v	0.4	Image HSV brightness enhancement

5.7. Experimental Results and Analysis

5.7.1. Effect of Different Model Improvement Methods on Training Results

The YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x models were selected for comparison of the evaluation results, as presented in Table 4.

Table 4. Comparison of evaluation metrics between different versions of YOLOv5 and YOLOv8.

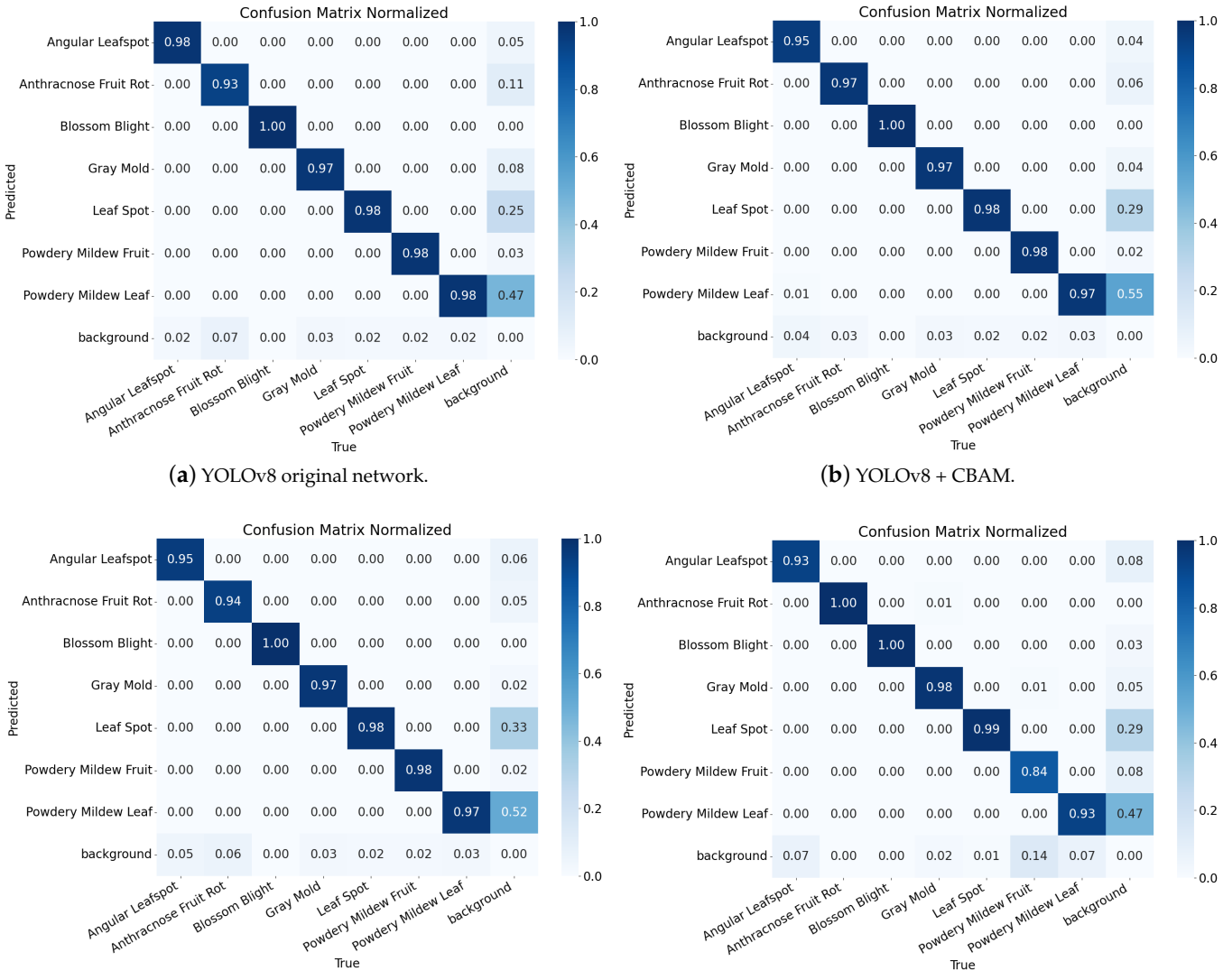
Model	Precision	Recall	*F*1	*mAP*@0.5	*mAP*@0.5:0.95	Inference Time/ms
YOLOv5s	0.96046	0.95615	0.9583	0.97645	0.88469	3.1
YOLOv5m	0.96664	0.96218	0.9644	0.98123	0.90865	3.5
YOLOv5l	0.96725	0.96919	0.96822	0.97649	0.90546	5.1
YOLOv5x	0.97657	0.96715	0.97184	0.98189	0.92103	7.6
YOLOv8s	0.96009	0.96018	0.96013	0.97884	0.9171	4.1
YOLOv8m	0.97633	0.95023	0.9631	0.98057	0.92874	4.6
YOLOv8l	0.97618	0.96267	0.96938	0.98313	0.93373	6.4
YOLOv8x	0.97036	0.96323	0.96678	0.98483	0.93849	8.7

In comparative analysis, YOLOv8 exhibited superior performance to YOLOv5 in terms of precision, recall, *F*1, and *mAP*. Additionally, the inference speed of YOLOv8-s was observed to be greater than that of other models within the YOLOv8 family due to its reduced number of parameters and lower computational complexity. To more effectively address the need for real-time detection of strawberry diseases, the YOLOv8-s network is being optimized for enhanced performance. Experiments were conducted to determine the impact of various improvement methods on the training results, where CBAM, DySample, and ODConv improvements were incorporated into the YOLOv8 network structure. These enhancements facilitated an improvement in feature representation, the capture of contextual information, and the enhanced performance of the YOLOv8 object recognition model. The results of the comparison experiments of the four models are presented in Table 5.

Table 5. Impact of different improvement methods on model performance.

Number	Model	Precision	Recall	*F*1	*mAP*@0.5	*mAP*@0.5:0.95
1	YOLOv8	0.96009	0.96018	0.96013	0.97884	0.9171
2	YOLOv8 + CBAM	0.97519	0.96194	0.96852	0.98257	0.91218
3	YOLOv8 + Dysample	0.98028	0.96161	0.97086	0.98043	0.92049
4	YOLOv8 + ODConv	0.95363	0.94999	0.95181	0.9709	0.91043

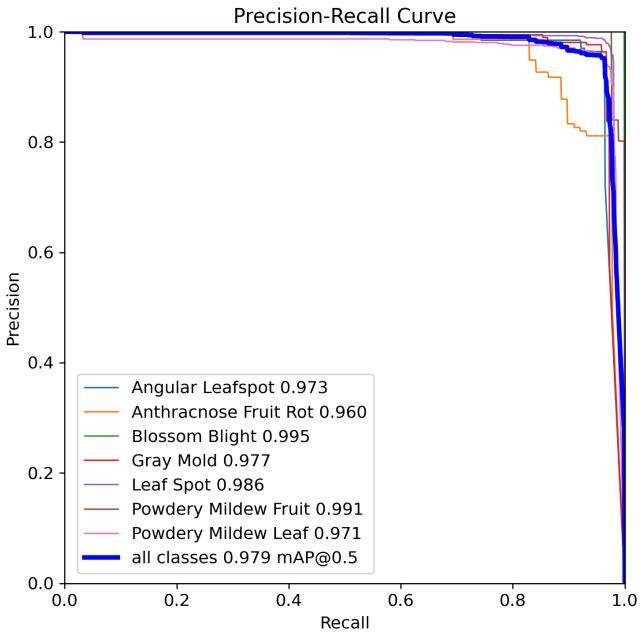
In Table 5, YOLOv8 + Dysample exhibits the highest $F1$ score, while YOLOv8 + CBAM has the highest $mAP@0.5$. These two enhancements demonstrate superior performance compared with several other attention mechanisms and are more suitable for strawberry pest and disease detection. The confusion matrices for the four models are presented in Figure 14, showing superior performance for all seven pests and diseases in the dataset. In Figure 15, the precision–recall curve is plotted in such a way that a model with a curve that is closer to the upper-right corner demonstrates superior performance, indicating higher recall while maintaining higher precision. To evaluate the performance of different models, the AUC can be applied as a metric. A higher AUC value indicates a more effective model. AUC values greater than 1 indicate superior performance. Figure 15 illustrates the precision–recall curves of the four models, where the YOLOv8 + CBAM and YOLOv8 + Dysample models have the greatest horizontal and vertical area under the curve, indicating superior detection accuracy and completeness across all detection categories. The results of Models 1 and 4 are somewhat inferior. In particular, the identification of Powdery Mildew Fruit and Anthracnose Fruit Rot samples is less accurate than that of Models 2 and 3.



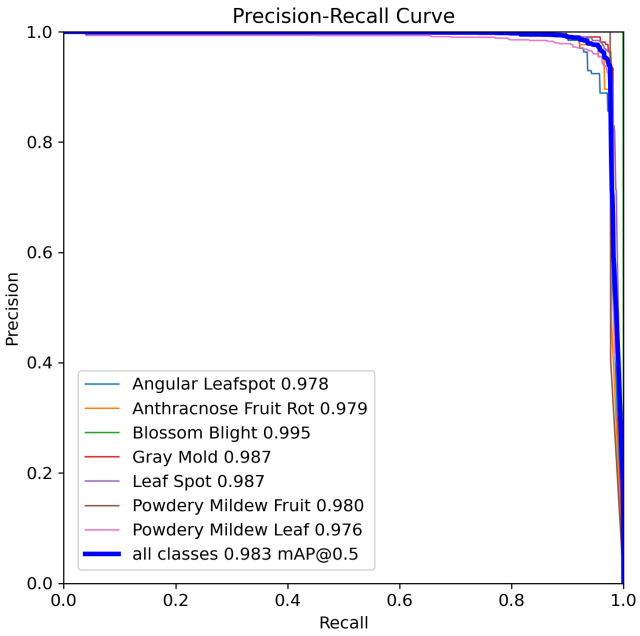
(**a**) YOLOv8 original network. (**b**) YOLOv8 + CBAM.

(**c**) YOLOv8 + Dysample. (**d**) YOLOv8 + ODConv.

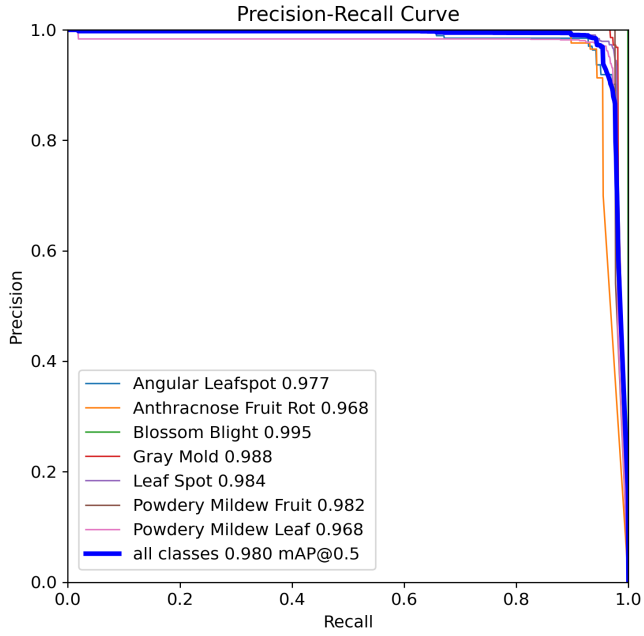
Figure 14. Confusion matrix for different improvement methods.



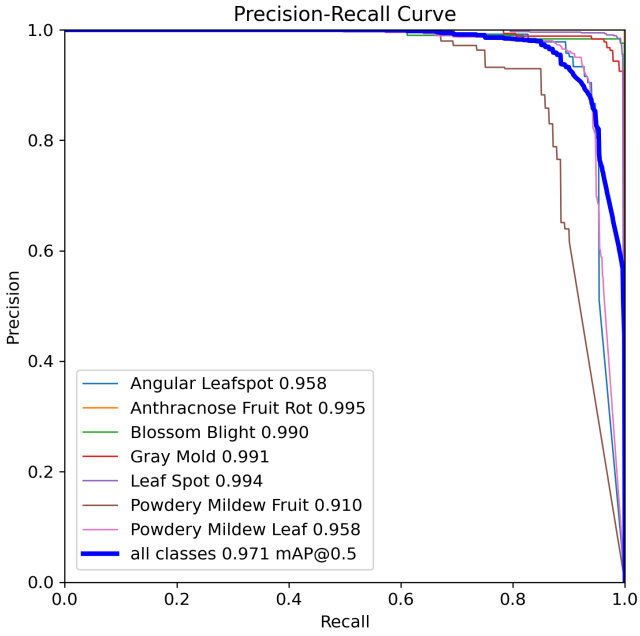
(**a**) YOLOv8 original network.



(**b**) YOLOv8 + CBAM.



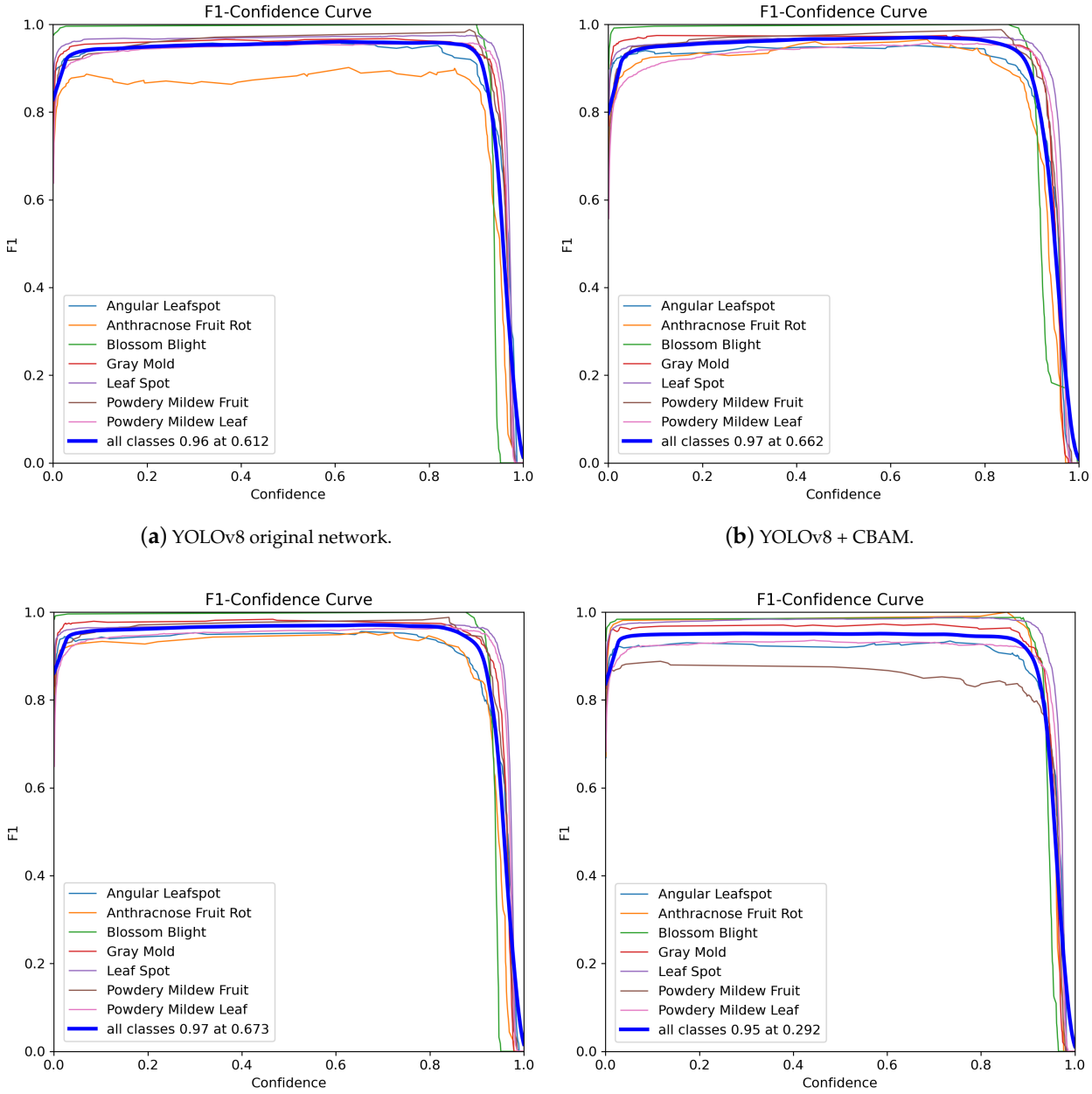
(**c**) YOLOv8 + Dysample.



(**d**) YOLOv8 + ODConv.

Figure 15. Precision–recall curves for different improved methods.

Figure 16 illustrates the *F*1 curves of various enhanced methodologies. The *F*1 curve can be used to assess the equilibrium between precision and recall of a model across a range of confidence thresholds. This enables the optimization of the model's performance. By analyzing the *F*1 curve, the confidence level that yields the highest *F*1 score can be identified. Figure 15 illustrates that the outcomes of Models 2 and 3 are superior to those of Models 1 and 4.



(a) YOLOv8 original network.

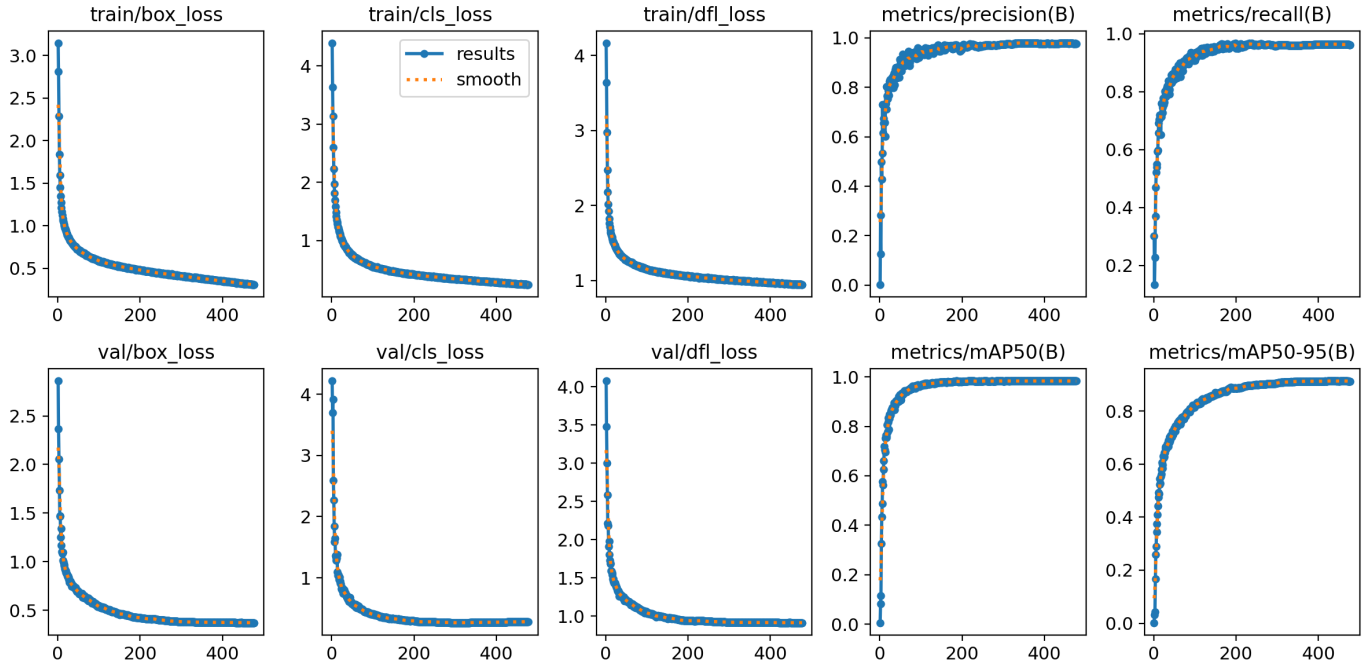
(b) YOLOv8 + CBAM.

(c) YOLOv8 + Dysample.

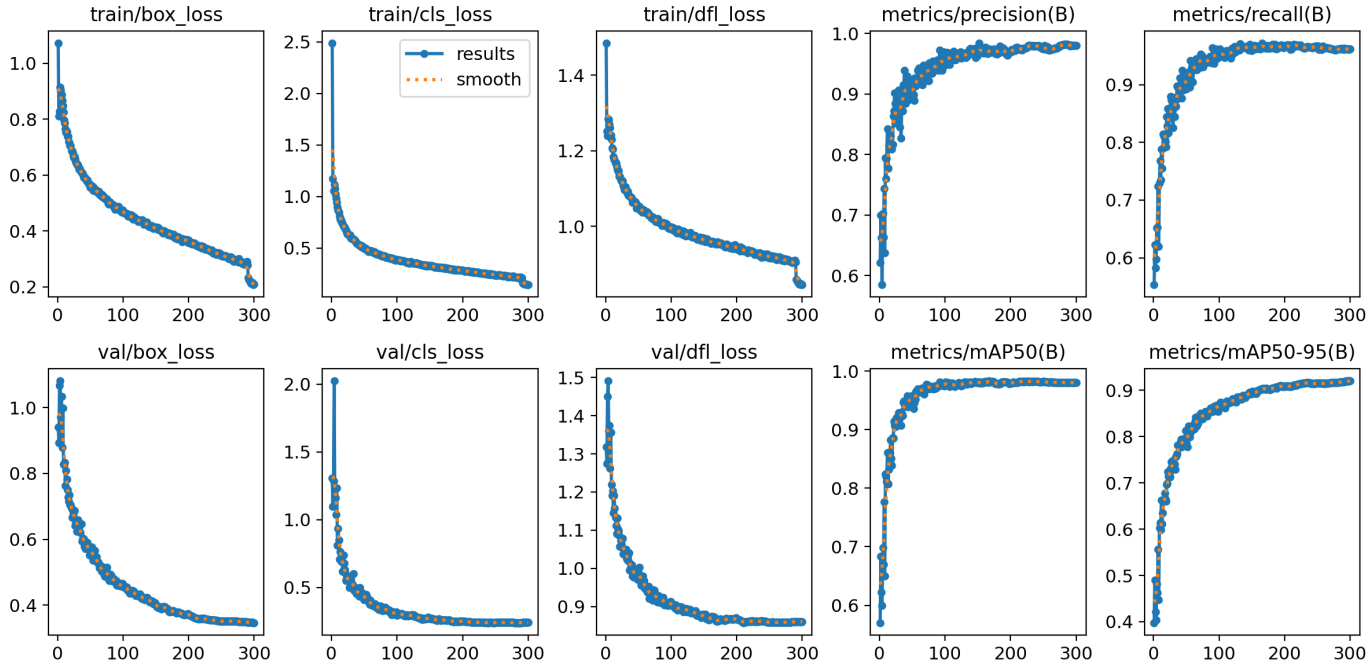
(d) YOLOv8 + ODConv.

Figure 16. *F*1 curves of different improved methods.

Figure 17 illustrates the comparative loss between Model 2 and Model 3. The loss function plays a pivotal role in target detection, evaluating the discrepancy between a model's predicted and actual values. The objective of box_loss is to minimize the localization loss, thereby enabling the model to accurately localize the target. Conversely, the purpose of class_loss is to minimize the classification loss, allowing the model to classify the target with greater precision. The curve of Model 2 is relatively stable, exhibiting minimal fluctuation and a more continuous trend of change. This indicates that the stability and convergence of model training are superior, and the change in indicators is relatively smooth. Figure 18 illustrates the real application of the YOLOv8 + CBAM model in the strawberry disease dataset.



(**a**) YOLOv8 + CBAM.



(**b**) YOLOv8 + Dysample.

Figure 17. Loss comparison of models.

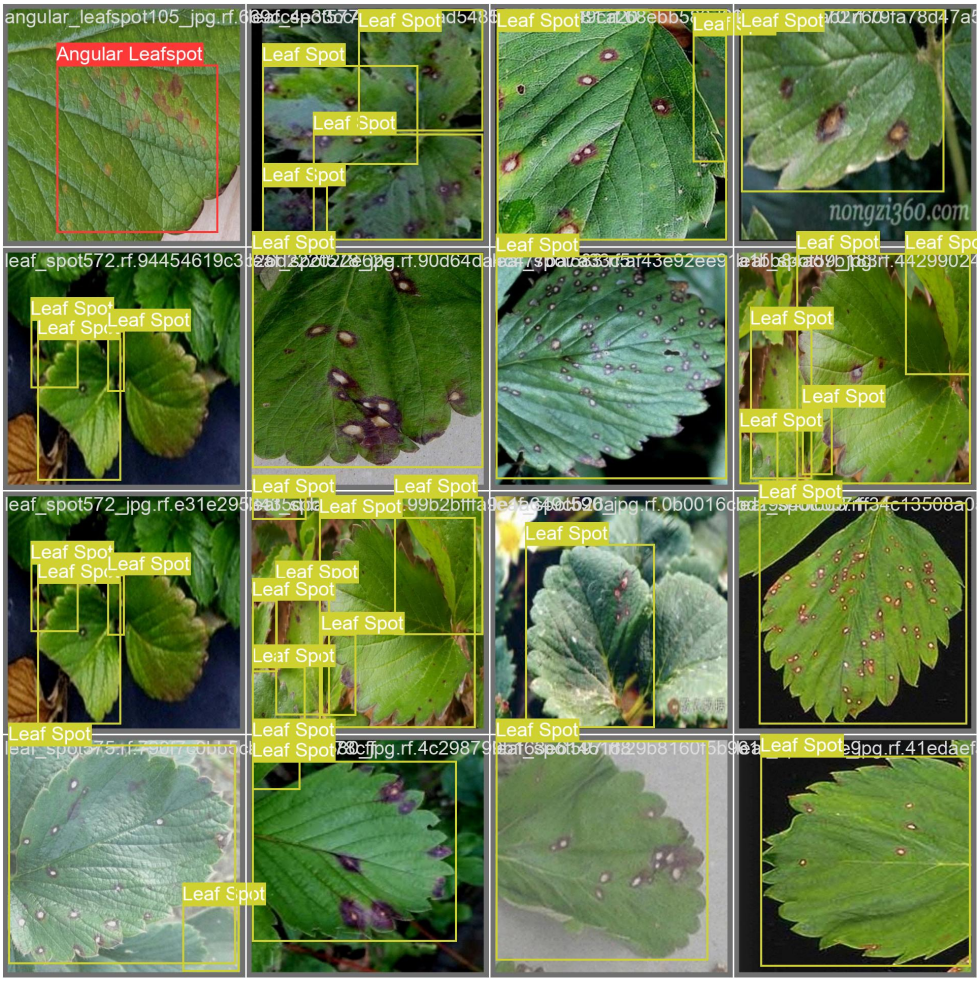


Figure 18. The real application of the YOLOv8 + CBAM model in the strawberry disease dataset.

5.7.2. Impact of TensorRT Acceleration on Model Efficiency

To enhance the efficacy of identifying pests and diseases affecting strawberries in real-time, we applied TensorRT acceleration. The PyTorch-trained models were converted to ONNX models. TensorRT was employed primarily to enhance the inference performance of deep learning models by leveraging the parallel computing power of GPUs to perform these operations in an efficient manner. TensorRT facilitates the optimization of network inference graphs, encompassing techniques such as graph clipping, layer fusion, and memory optimization. To facilitate the implementation of the model in cloud and or edge devices, quantization techniques were utilized to optimize memory and reduce the requisite computational resources [34,35]. The weights and activations of the network are represented by lower-precision data types. In this instance, three common lower-precision data types are utilized: FP16, FP32, and INT8. The inference times are illustrated in Table 6.

Table 6. Comparisons of inference time for different quantification methods.

Model	Fp32/ms	Fp16/ms	Int8/ms
YOLOv8	1.33	0.61	0.44
YOLOv8 + CBAM	1.33	0.61	0.44
YOLOv8 + Dysample	1.36	0.71	0.56

The inference time for FP16 quantization with TensorRT was selected and compared to the case without TensorRT. Experiments were conducted to compare the average reasoning time before and after acceleration of the three models: the original YOLOv8 model, the

modified version including the CBAM attention mechanism, and the modified version including Dysample. The basis for comparison was the time taken for CPU or GPU computation. The time taken for GPU inference was automatically recorded by parsing ONNX when the engine file was generated. The results are presented in Table 7. It is evident that the average inference time per image is in excess of 10 ms prior to acceleration, whereas subsequent to acceleration with TensorRT, the inference time was reduced to approximately 3 ms. TensorRT engine acceleration effectively reduced the inference time to approximately one-third of the original time, which in turn reduced computational latency, saved computational resources, and improved real-time performance. This work facilitates the identification of strawberry pests and diseases by image frames or alternative frames during subsequent automated detection by capturing videos of the pests and diseases using a high-definition camera. This approach enhances automation and intelligence.

Table 7. Impact of TensorRT acceleration on inference time.

Model	Average Inference Time before Acceleration/ms	Average Inference Time after Acceleration/ms
YOLOv8	4.1	0.61
YOLOv8 + CBAM	5.2	0.61
YOLOv8 + Dysample	4.9	0.71

6. Conclusions

We proposed a real-time method for the detection of pests and diseases affecting strawberries. Through a series of model comparisons, we recommend the use of the YOLOv8 + CBAM or YOLOv8 + Dysample model, in conjunction with TensorRT acceleration, for this purpose. These methods optimally balance between accuracy, speed, and computational requirements and demonstrate superior performance compared with current state-of-the-art real-time object detection algorithms. The enhanced YOLOv8 model demonstrates superior detection accuracy while maintaining high processing speeds. To preserve the integrity of the extensive pest and disease dataset, minimal preprocessing was conducted. Model detection results did not indicate that an imbalance in the dataset resulted in a greater bias toward categories with more training samples. The distribution of the dataset reflects the actual prevalence of pests and diseases in the natural environment. Currently, the improved YOLOv8 model has been deployed in the cloud and accessed. The developed mobile application facilitates the real-time detection of strawberry pests and diseases via the application programming interface (API) calls in the field. Currently, our model is able to identify seven common strawberry pests and diseases, and there are others for which the model has not been trained. For early strawberry pests and diseases, the model must be improved and refined due to the lack of distinctive characteristics of pests and possible similarities between diseases.

Author Contributions: Conceptualization, D.X.; methodology, D.X. and Z.S.; resources, D.X. and W.Y.; software, D.X. and W.Y.; validation, W.Y. and W.S.; data curation, D.X. and Z.S.; writing—original draft preparation, D.X.; formal analysis, Z.S.; investigation, W.Y. and W.S.; writing—review and editing, D.X. and Z.S.; project administration, D.X. and Z.S.; supervision, Z.S.; funding acquisition, D.X. and Z.S. All authors have read and agreed to the published version of the manuscript.

Funding: This study was partially supported by the Qinglan Project of Jiangsu Universities, the Jiangsu Province Science and Technology Vice General Project (No. FZ20231155), the Talent Development Project of Taizhou University (No. TZXY2018QDJJ006), and the Young Science and Technology Talent Support Project of Taizhou.

Data Availability Statement: All data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

YOLOv5	You Only Look Once version 5
YOLOv8	You Only Look Once version 8
SOTA	State-of-the-Art
CBAM	Convolutional Block Attention Module
CAM	Channel Attention Module
SAM	Spatial Attention Module
DySample	Super-Lightweight Dynamic Upsampling Operator
ODConv	Omni-Dimensional Dynamic Convolution
CNNs	Convolutional Neural Networks
VGGNet	Visual Geometry Group Network
ResNet	Residual Network
DenseNet	Dense Convolutional Network
TensorRT	Tensor Routing and Transposing
AP	Average Precision
mAP	Mean Average Precision
ONNX	Open Neural Network Exchange
AUC	Area Under the Curve
FP32	32-Bit Floating Point
FP16	Half-Precision Floating Point
INT8	8-Bit Integer

References

1. Liu, J.; Wang, X. Plant diseases and pests detection based on deep learning: A review. *Plant Methods* **2021**, *17*, 22 . [CrossRef] [PubMed]
2. Chai, J.; Zeng, H.; Li, A.; Ngai, E.W. Deep learning in computer vision: A critical review of emerging techniques and application scenarios. *Mach. Learn. Appl.* **2021**, *6*, 100134. [CrossRef]
3. Torfi, A.; Shirvani, R.A.; Keneshloo, Y.; Tavaf, N.; Fox, E.A. Natural language processing advancements by deep learning: A survey. *arXiv* **2020**, arXiv:2003.01200.
4. Pushpa, B.R.; Ashok, A.; Hari, A.V.S. Plant disease detection and classification using deep learning model. In Proceedings of the 2021 Third International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, 2–4 September 2021 ; pp. 1285–1291.
5. Ahmad, A.; Saraswat, D.; El Gamal, A. A survey on using deep learning techniques for plant disease diagnosis and recommendations for development of appropriate tools. *Smart Agric. Technol.* **2023**, *3*, 100083. [CrossRef]
6. Paymode, A.S.; Malode, V.B. Transfer learning for multi-crop leaf disease image classification using convolutional neural network VGG. *Artif. Intell. Agric.* **2022**, *6*, 23–33. [CrossRef]
7. Hu, W.J.; Fan, J.; Du, Y.X.; Li, B.S.; Xiong, N.; Bekkering, E. MDFC–ResNet: An agricultural IoT system to accurately recognize crop diseases. *IEEE Access* **2020**, *8*, 115287–115298. [CrossRef]
8. Thakur, P.S.; Chaturvedi, S.; Khanna, P.; Sheorey, T.; Ojha, A. Vision transformer meets convolutional neural network for plant disease classification. *Ecol. Inform.* **2023**, *77*, 102245. [CrossRef]
9. Ashwini, C.; Sellam, V. An optimal model for identification and classification of corn leaf disease using hybrid 3D-CNN and LSTM. *Biomed. Signal Process. Control* **2024**, *92*, 106089. [CrossRef]
10. Zhang, W.; Sun, Y.; Huang, H.; Pei, H.; Sheng, J.; Yang, P. Pest region detection in complex backgrounds via contextual information and multi-scale mixed attention mechanism. *Agriculture* **2022**, *12*, 1104. [CrossRef]
11. Yang, S.; Xing, Z.; Wang, H.; Dong, X.; Gao, X.; Liu, Z.; Zhao, Y. Maize-YOLO: A new high-precision and real-time method for maize pest detection. *Insects* **2023**, *14*, 278. [CrossRef]
12. Song, L.; Liu, M.; Liu, S.; Wang, H.; Luo, J. Pest species identification algorithm based on improved YOLOv4 network. *Signal Image Video Process.* **2023**, *17*, 3127–3134. [CrossRef]
13. Appe, S.N.; Arulselvi, G.; Balaji, G.N. CAM-YOLO: Tomato detection and classification based on improved YOLOv5 using combining attention mechanism. *Peerj Comput. Sci.* **2023**, *9*, 1463. [CrossRef] [PubMed]
14. Tian, Y.; Yang, G.; Wang, Z.; Li, E.; Liang, Z. Detection of apple lesions in orchards based on deep learning methods of CycleGAN and YOLOV3-dense. *J. Sens.* **2019**, *1*, 7630926. [CrossRef]
15. Wani, J.A.; Sharma, S.; Muzamil, M.; Ahmed, S.; Sharma, S.; Singh, S. Machine learning and deep learning based computational techniques in automatic agricultural diseases detection: Methodologies, applications, and challenges. *Arch. Comput. Methods Eng.* **2022**, *29*, 641–677. [CrossRef]

16. Singh, V.; Misra, A.K. Detection of plant leaf diseases using image segmentation and soft computing techniques. *Inf. Process. Agric.* **2017**, *4*, 41–49. [CrossRef]
17. Griffel, L.M.; Delparte, D.; Edwards, J. Using Support Vector Machines classification to differentiate spectral signatures of potato plants infected with Potato Virus Y. *Comput. Electron. Agric.* **2018**, *153*, 318–324. [CrossRef]
18. Kartikeyan, P.; Shrivastava, G. Review on emerging trends in detection of plant diseases using image processing with machine learning. *Int. J. Comput. Appl.* **2021**, *975*, 39–48. [CrossRef]
19. Yu, H.; Liu, J.; Chen, C.; Heidari, A.A.; Zhang, Q.; Chen, H. Optimized deep residual network system for diagnosing tomato pests. *Comput. Electron. Agric.* **2022**, *195*, 106805. [CrossRef]
20. Abbas, I.; Liu, J.; Amin, M.; Tariq, A.; Tunio, M.H. Strawberry fungal leaf scorch disease identification in real-time strawberry field using deep learning architectures. *Plants* **2021**, *10*, 2643. [CrossRef]
21. Perez-Borrero, I.; Marin-Santos, D.; Vasallo-Vazquez, M.J.; Gegundez-Arias, M.E. A new deep-learning strawberry instance segmentation methodology based on a fully convolutional neural network. *Sneural Comput. Appl.* **2021**, *33*, 15059–15071. [CrossRef]
22. Yang, G.-F.; Yong, Y.; He, Z.-K.; Zhang, X.-Y.; He, Y. A rapid, low-cost deep learning system to classify strawberry disease based on cloud service. *J. Integr. Agric.* **2022**, *21*, 460–473.
23. Sohan, M.; Sai Ram, T.; Reddy, R.; Venkata, C. A review on yolov8 and its advancements. In *International Conference on Data Intelligence and Cognitive Informatics*; Springer: Singapore, 2024; pp. 529–545.
24. Wang, C.; Sun, S.; Zhao, C.; Mao, Z.; Wu, H.; Teng, G. A detection model for cucumber root-knot nematodes based on modified YOLOv5-CMS. *Agronomy* **2022**, *12*, 2555. [CrossRef]
25. Yang, W.; Qiu, X. A lightweight and efficient model for grape bunch detection and biophysical anomaly assessment in complex environments based on YOLOv8s. *Front. Plant Sci.* **2024**, *15*, 1395796. [CrossRef] [PubMed]
26. Ye, R.; Shao, G.; Yang, Z.; Sun, Y.; Gao, Q.; Li, T. Detection Model of Tea Disease Severity under Low Light Intensity Based on YOLOv8 and EnlightenGAN. *Plants* **2024**, *13*, 1377. [CrossRef] [PubMed]
27. Xiao, B.; Nguyen, M.; Yan, W.Q. Fruit ripeness identification using YOLOv8 model. *Multimed. Tools Appl.* **2024**, *83*, 28039–28056. [CrossRef]
28. Jia, R.; Lv, B.; Chen, J.; Liu, H.; Cao, L.; Liu, M. Underwater Object Detection in Marine Ranching Based on Improved YOLOv8. *J. Mar. Sci. Eng.* **2023**, *12*, 55. [CrossRef]
29. Woo, S.; Park, J.; Lee, J.Y.; Kweon, I.S. Cbam: Convolutional block attention module. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 3–19.
30. Liu, W.; Lu, H.; Fu, H.; Cao, Z. Learning to upsample by learning to sample. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Paris, France, 2–3 October 2023; pp. 6027–6037.
31. Ma, J.; Zhang, Z.; Xiao, W.; Zhang, X.; Xiao, S. Flame and smoke detection algorithm based on ODConvBS-YOLOv5s. *IEEE Access* **2023**, *11*, 34005–34014. [CrossRef]
32. Yadav, S.; Shukla, S. Analysis of k-fold cross-validation over hold-out validation on colossal datasets for quality classification. In Proceedings of the 2016 IEEE 6th International Conference on Advanced Computing (IACC), Bhimavaram, India, 27–28 February 2016; pp. 78–83.
33. Dewi, C.; Chen, R.C.; Zhuang, Y.C.; Jiang, X.; Yu, H. Recognizing road surface traffic signs based on YOLO models considering image flips. *Big Data Cogn. Comput.* **2023**, *7*, 54. [CrossRef]
34. Shafique, M.A.; Munir, A.; Kong, J. Deep Learning Performance Characterization on GPUs for Various Quantization Frameworks. *AI* **2023**, *4*, 926–948. [CrossRef]
35. Li, Z.; Li, H.; Meng, L. Model Compression for Deep Neural Networks: A Survey. *Computers* **2023**, *12*, 60. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.